

The Impact of Self-Distillation on Minimal Compute Cost for Target Pre-training Loss in LLMs

Luka Ivanić*, Denis Ibišić* and Teo Verbanac*

Abstract—The performance of large language models (LLMs) is intrinsically linked to their scale, but the escalating computational costs present significant sustainability challenges. This research investigates whether self-distillation can reduce the compute cost required to achieve a target pre-training loss. We perform a comparative analysis of three training paradigms—classical, teacher-student knowledge distillation, and self-distillation—across a spectrum of model sizes from 1.7M to 154M parameters. Our results indicate that while classical distillation acts as a potent convergence accelerator, its effectiveness is ultimately bounded by the student model’s capacity. Contrary to our primary hypothesis, our self-distillation implementation failed to improve performance and resulted in a higher training loss than the classical baseline.

I. INTRODUCTION

Progress in the performance of LLMs is strongly linked to their size, as demonstrated in examples such as GPT-1 [1], BERT [2], Megatron-LM [3], and GPT-3 [4]. Empirical evidence indicates that model performance follows predictable scaling laws, where test loss is a function of model size, dataset size, and computational cost [5]. An empirical formula was also derived to estimate the minimum computational cost required to achieve a target test loss, assuming an optimal model size and a sufficiently large dataset [5]. This relationship is given by eq. (1) where the empirically derived constants are $C_{c_{\min}} \approx 3.1 \cdot 10^8$ PF-days and $\alpha \approx 0.050$, and where $C_{\min}$ represents the minimum number of PF-days invested in training.

$$L(C_{\min}) = \left(\frac{C_{c_{\min}}}{C_{\min}} \right)^{\alpha} \quad (1)$$

Although these scaling laws allow for the reliable prediction of computational costs for developing better and larger LLMs, the exponential nature of these costs in relation to performance raises questions about sustainability and the potential of reaching the physical limitations of current hardware like GPUs and TPUs. To mitigate this issue, various methods for reducing the training and inference costs of LLMs are being explored, with knowledge distillation emerging as one of the most promising approaches.

Knowledge distillation, which involves transferring knowledge from larger teacher models to student models with significantly fewer parameters, has proven to be a highly effective method for improving the performance and accelerating the training of smaller models, as shown in works like DistilBERT [6], TinyBERT [7] and MiniLLM [8]. Additionally, recent research has demonstrated that LLMs possess the

capability for self-improvement [9], [10]. This raises a key question: can a model use its deeper layers to teach earlier layers, accelerating convergence and lowering compute cost below the predicted optimum?

This work hypothesizes that self-distillation can accelerate model convergence, thereby achieving target performance with lower computational cost compared to classically trained models.

The aim of this work is to investigate the effects of various knowledge distillation methods on LLMs, analyzing them across a wide range of model sizes and input sequence lengths. We compare the performance of models trained using a classical approach, based on established best practices, against those trained using knowledge distillation techniques. Our analysis is structured in three parts: First, we establish performance baselines by training a range of models using a classical approach. Second, we investigate the effectiveness of knowledge distillation, where smaller “student” models learn from a larger, pre-trained “teacher”. Finally, we explore self-distillation, analyzing its impact on convergence speed and performance when a model learns from its own internal representations. This comparative study aims to quantify the benefits of each approach in the context of minimizing computational cost for a target pre-training loss.

The paper is structured as follows. Section II provides a comprehensive overview of our methodology, from data preparation and model architecture to the implementation of the knowledge distillation frameworks. Section III specifies the case study design and details the evaluation metrics we used. In Section IV, we present the results of our comparative experiments. We then discuss the implications of these findings in Section V and provide directions for future research, and conclude the paper in Section VI with a summary of our contributions.

II. METHODOLOGY

This section outlines the complete methodology used in our research, from data acquisition and preprocessing to model architecture, the knowledge distillation framework, and the training and inference procedures.

A. Dataset and Preprocessing

Our work is based on the Wikitext-103-v1 dataset, used in prior LLM studies such as GPT-1 [1] and BERT [2]. The dataset is a large, high-quality corpus of English text derived from verified “Good” and “Featured” articles on Wikipedia, making it a suitable and challenging benchmark for causal

*M.Sc. Student in Software Engineering, Department of Computer Science, Faculty of Engineering, University of Rijeka, Rijeka, Croatia

language modeling. We prepare the dataset through a two-stage process: we first train a Byte-Pair Encoding (BPE) tokenizer and then implement a data pipeline to transform the raw text into fixed-length training examples.

- **Tokenizer Generation:** We train a custom BPE tokenizer from scratch on the training split of the dataset. The tokenizer employs a ByteLevel pre-tokenizer for robustness to any input text and applies NFKC Unicode normalization. It is configured with a vocabulary size of 5,000 and contains special tokens for unknown words ([UNK]), padding ([PAD]), End-of-Sequence ([CLS]) and Beginning-of-Sequence ([SEP]).
- **Data Pipeline:** For causal language model training, the raw text articles undergo a sequential transformation. Each article is first tokenized, with ([SEP]) and ([CLS]) tokens appended. All tokenized articles are then concatenated into a single, continuous stream of token IDs. To preserve document boundaries from the raw Wikitext-103 dataset, our pipeline uses regular expressions to identify section titles (e.g., `= Section Title =`) as delimiters, ensuring that semantically distinct articles are not arbitrarily merged before being segmented. The stream is subsequently segmented into fixed-length blocks matching the model’s context length. For the causal modeling task, the input sequence for each block is used directly, while the target labels are created by shifting the same sequence one position to the right. Any final, shorter sequence is discarded from the dataset.

B. Model Architecture

We design and implement a decoder-only Transformer model from the ground up. The architecture is defined by a stack of identical Transformer blocks, preceded by embedding layers and followed by a final language model head.

- **Embeddings:** The model uses two embedding layers: a standard token embedding layer and a learnable positional embedding layer, as introduced in the original Transformer architecture [11]. Their outputs are summed to provide the input to the first Transformer block.
- **Transformer Block:** Each block consists of two main sub-layers:
 - 1) A multi-head self-attention (MHA) mechanism that computes a weighted sum over all input values. The weights are determined by the similarity between a *query* from the current token and the *keys* of all other tokens. This process is performed in parallel across multiple ‘heads’ to capture diverse contextual relationships. An optimized implementation leverages FlashAttention [12] for computational efficiency. Finally, dropout is applied to the attention scores.
 - 2) A position-wise feed-forward network (FFN), consisting of two linear layers with a GELU activation

function, as used in BERT [2] and Megatron-LM [3]. The hidden dimension of the FFN is four times the model’s primary dimension. A dropout layer is applied to its output.

The architecture follows a Pre-LN configuration: both sub-layers are preceded by a normalization layer and followed by a residual connection.

- **LM Head:** The final output layer projects the transformer’s final hidden state back into the vocabulary space to generate token logits. This layer effectively functions as a “de-embedding” layer, reversing the initial token embedding process. Its weights are tied with the token embedding matrix, following strategies used in models like BERT [2] and T5 [13], to reduce the total number of parameters and improve performance.
- **Weight Initialization:** We employ a specific weight initialization scheme to enhance training stability. All weights are initialized from a normal distribution with a mean of 0 and a standard deviation of $d_{\text{model}}^{-0.5}$, where d_{model} represents the dimensionality of the model’s vector representations. Furthermore, we apply a scaled residual initialization strategy. The output projection weights in the multi-head self-attention (MHA) and the second linear layer in the FFN are scaled by a factor of $1/\sqrt{2 \cdot N_{\text{layers}}}$, with N_{layers} being the total number of Transformer blocks. This ensures that at the start of training, the contribution of each Transformer block is minimal, forcing the information to flow primarily through the residual pathway. This encourages the model to learn transformations gradually, preventing the variance of activations from growing and maintaining stability in deep models.

C. Training paradigms

Our research evaluates three distinct training paradigms to analyze their impact on model performance and computational efficiency. We implement a classical baseline training approach, a knowledge distillation framework using an external teacher model, and a self-distillation mechanism for intra-model knowledge transfer.

- **Classical Training:** The baseline for our experiments is established through a classical training procedure heavily informed by the principles of compute-optimal training and scaling laws outlined in seminal works by Kaplan et al. [5] and Hoffmann et al. [14]. Models are trained from scratch to minimize the cross-entropy loss between their predictions and the target tokens. Rather than strictly adhering to a fixed recipe, we conduct a series of experiments to determine the final, optimized hyperparameters for each model size, as presented in Table I. This empirical approach ensures that each baseline model is robust and serves as a fair benchmark for comparison.
- **Knowledge Distillation (Teacher-Student):** This paradigm involves transferring knowledge from a large, powerful, pre-trained teacher model (selected from classical training baseline) to a smaller, more efficient

student model. The student model is then trained to minimize a combined loss function as shown in eq. (2), where $\mathcal{L}_{\text{CE}}$ is the standard cross-entropy loss on the ground-truth labels, and $\mathcal{L}_{\text{distill}}$ is the distillation loss, which encourages the student to mimic the teacher.

$$\mathcal{L}_{\text{total}} = (1 - \alpha)\mathcal{L}_{\text{CE}} + \alpha \cdot \mathcal{L}_{\text{distill}} \quad (2)$$

The hyperparameter α balances the two objectives. To balance the contributions of the distillation and cross-entropy losses during training, we implement a dynamic normalization scheme. Instead of using a fixed weighting factor, we rescale the distillation loss ($\mathcal{L}_{\text{distill}}$) in each training step so that its mean value over the batch matches the mean of the cross-entropy loss ($\mathcal{L}_{\text{CE}}$). This prevents one loss from dominating the gradient updates. The modified distillation loss, $\mathcal{L}'_{\text{distill}}$, is calculated as shown in eq. (3), where $\overline{\mathcal{L}_{\text{CE}}}$ and $\overline{\mathcal{L}_{\text{distill}}}$ represent the mean values of the respective losses for the current batch. The total loss for the training step is then the sum of $\mathcal{L}'_{\text{distill}}$ and $\mathcal{L}_{\text{CE}}$.

$$\mathcal{L}'_{\text{distill}} = \mathcal{L}_{\text{distill}} \times \frac{\overline{\mathcal{L}_{\text{CE}}}}{\overline{\mathcal{L}_{\text{distill}}}} \quad (3)$$

Our framework supports several distillation modes:

- 1) **Logits Distillation:** The student learns to match the teacher’s output probability distribution by minimizing the Kullback-Leibler (KL) Divergence between their softened logits.
- 2) **Hidden State and Attention Distillation:** The student can also be trained to replicate the teacher’s intermediate hidden states (via Mean Squared Error (MSE) loss) or attention patterns (via KL Divergence), transferring richer hierarchical knowledge.
- **Self-Distillation** In this approach, a model learns from its own internal representations without an external teacher. We investigate a specific form of this paradigm where the teacher is a frozen, earlier checkpoint of the model itself. This creates a teacher-student dynamic where the student learns from a more stable and knowledgeable version of itself. The core mechanism involves transferring knowledge from the deeper, more complex layers of the frozen teacher to the shallower layers of the student. The distillation loss $\mathcal{L}_{\text{distill}}$ is defined based on the chosen mode:

$$\mathcal{L}_{\text{distill}} = \text{MSE}(H_{\text{student}}^{(i)}, H_{\text{teacher}}^{(j)}) \quad (4)$$

- 2) **Attention Distillation:** A shallower student layer i is trained to replicate the attention probability

distribution A of a deeper teacher layer j using KL Divergence.

$$\mathcal{L}_{\text{distill}} = D_{\text{KL}}(A_{\text{student}}^{(i)} \| A_{\text{teacher}}^{(j)}) \quad (5)$$

In our implementation, the weighting coefficient α is either a fixed value for MSE-based distillation or linearly scaled with the training step for KL divergence. The indices of the teacher (j) and student (i) layers are configurable hyperparameters.

D. Training Procedure

The specific hyperparameters for each model are centrally configured and detailed in Table I. The main components of our training procedure include:

- **Optimizer and Scheduler:** We use the AdamW optimizer with a weight decay of 0.1. While larger models like T5 have successfully utilized optimizers such as AdaFactor to manage the large memory footprint of the optimizer states [13], AdamW remains a highly effective and standard choice for the model sizes explored in this work. The learning rate follows a schedule with a linear warmup phase over the first 2,000 steps, followed by a cosine decay over the total 12,000 training steps. A graphical representation of this learning rate schedule is provided in Appendix I (Fig. 7). It is worth noting that while specific learning rates are reported in Table I, our preliminary sensitivity analysis indicates a high degree of robustness in the training outcomes with respect to the learning rate schedule. We observe that as long as the learning rate is kept within a reasonable order of magnitude, the combination of the AdamW optimizer and a cosine decay schedule consistently guide the models to a similar final performance level. This finding, which mitigates the need for exhaustive hyperparameter tuning, highlights the stability of modern optimization techniques. The graphical results of this analysis are presented in Appendix III.
- **Precision and Hardware:** Training is performed on a CUDA-enabled GPU. We leverage Automatic Mixed Precision (AMP) to accelerate training, with support for both `bfloat16` and `float16` data types. When using `float16`, a gradient scaler is employed to prevent underflow issues with small gradients.
- **Regularization:** In addition to dropout, we apply gradient clipping by norm, as done in Megatron-LM [3] and also applied in GPT-1 [1] and GPT-3 training [4], to prevent exploding gradients during training.
- **Experiment Tracking:** Reproducibility and detailed analysis are ensured through deep integration with Weights & Biases (W&B). Our pipeline automatically logs all hyperparameters, training and validation metrics. Model checkpoints are versioned and saved as W&B artifacts. We leverage the PyTorch profiler to capture detailed performance traces which are logged as well to W&B.

TABLE I
KEY HYPERPARAMETERS FOR CLASSICAL MODEL TRAINING ACROSS SIZES

Model	#Params	Emb.	Layers	Heads	Ctx.	Batch	LR _{max}	LR _{min}
1M Classic	1.7M	128	5	8	512	16k	$1.0e-3$	$1.0e-4$
2M Classic	2.89M	128	11	8	512	16k	$1.0e-3$	$1.0e-4$
5M Classic	6.28M	256	6	8	1024	16k	$1.0e-3$	$1.0e-4$
10M Classic	14.9M	256	13	8	1024	16k	$8.0e-4$	$8.0e-5$
30M Classic	34.59M	512	10	16	1024	32k	$8.0e-4$	$8.0e-5$
70M Classic	75.56M	512	23	16	1024	32k	$6.0e-4$	$6.0e-5$
150M Classic	154.33M	512	48	16	1024	32k	$3.0e-4$	$3.0e-5$

E. Text Generation Strategy

To assess the qualitative output of our trained models, we employ a controlled text generation strategy that combines several sampling techniques to produce coherent and diverse text. Our approach is designed to mitigate common failure modes of autoregressive generation, such as repetitive loops and deterministic, uninteresting output.

The core of our inference pipeline utilizes a combination of Top-k and Top-p (Nucleus) sampling, augmented with temperature scaling and a repetition penalty.

- **Temperature Scaling:** We apply a temperature T to the model’s output logits to control the randomness of the predictions. Lower temperatures make the distribution sharper and generation more deterministic, while higher temperatures flatten the distribution, increasing diversity. This is implemented by dividing each logit value by the temperature T before the softmax function is applied.
- **Top-p (Nucleus) Sampling:** We restrict the sampling pool to the smallest set of tokens whose cumulative probability exceeds a certain threshold p . This adapts the number of considered tokens dynamically based on the model’s confidence. In practice, tokens are first sorted by probability, and those outside the cumulative threshold p are removed from the candidate pool by setting their logits to negative infinity.
- **Top-k Sampling:** The final selection is made from the top k most probable tokens, further constraining the choices to a fixed-size set of high-quality candidates. This is implemented by selecting the k tokens with the highest scores, after which the probability distribution is recalculated (renormalized) over only this reduced set before the final token is sampled.
- **Repetition Penalty:** To discourage the model from getting stuck in repetitive loops, we apply a penalty to the logits of tokens that have already appeared in the generated sequence. This is achieved by identifying all unique tokens in the existing output and reducing their corresponding logits by a fixed penalty factor, thus making them less likely to be chosen again.

III. CASE STUDY

This section details the experimental setup for our case study, including the dataset composition, data partitioning strategy, and the metrics used to evaluate the performance and efficiency of our self-distillation framework.

A. Dataset

To provide a clear understanding of the dataset’s scale and composition, its raw statistics are presented in Table II.

TABLE II
WIKITEXT-103-v1 DATASET STATISTICS

Statistic	Training	Validation	Test
Number of Articles	28,475	60	60
Number of Lines	2.3M	4.9k	5.8k
Number of Words	101.4M	213.9k	241.2k
Number of Characters	539.5M	1.1M	1.3M
Vocabulary Size	547.4k	17.6k	18.8k
Avg. Words per Article	3,625	3,627	4,092

A critical step in our data processing pipeline is the selection of the tokenizer’s vocabulary size, as it presents a fundamental trade-off between the model’s representational capacity and the overall size of the training corpus. To quantify this trade-off, we trained and analyzed a range of BPE tokenizers with varying vocabulary sizes on the Wikitext-103-v1 training set. The results, provided in Table III, demonstrate a clear inverse relationship between vocabulary size and token-to-word ratio.

As demonstrated in Table III a clear inverse relationship exists between the vocabulary size and the token-to-word ratio. This tokenization efficiency carries significant implications. A lower ratio effectively increases the model’s semantic context window; for a fixed number of input tokens, the model can process a larger volume of actual text, which is crucial for understanding long-range dependencies.

TABLE III
COMPARISON OF TOKENIZATION EFFICIENCY FOR DIFFERENT
VOCABULARY SIZES ON THE WIKITEXT-103 TRAINING SET

Vocabulary Size	T-t-W Ratio	Total Tokens	Training Blocks
2,000	1.7348	179,431,233	87,613
5,000	1.4361	149,313,140	72,907
8,000	1.3206	137,435,572	67,108
12,000	1.2411	129,317,896	63,144

However, this efficiency introduces a critical architectural trade-off, especially for the smaller models targeted in this study. The vocabulary size directly dictates the number of parameters in the token embedding and language model head layers. The number of parameters in the embedding and

output layer is given by eq. (6) where V is the vocabulary size and d is the hidden dimension.

$$\text{Number of embedding parameters} = V \cdot d \quad (6)$$

With a large vocabulary, these layers can disproportionately dominate the model’s total parameter count. For instance, in a 1M parameter model, a large vocabulary could result in an embedding matrix that is significantly larger than all the computational Transformer blocks combined. Such a model would function more like a large lookup table with limited processing capacity.

Given our model’s architecture with a hidden dimension of 256, a vocabulary of 12,000 would allocate over 3 million parameters to the tied token embedding and language model head layers alone, exceeding the parameter count of the Transformer blocks responsible for computation.

Therefore, for our subsequent experiments, we selected the vocabulary size of 5,000. This choice represents a deliberate trade-off, balancing reasonable compression efficiency against the need to maintain a lean parameter budget appropriate for small-scale language models.

B. Evaluation

To provide a comprehensive assessment of our model, we employ a combination of metrics that evaluate both its language modeling capabilities and its computational efficiency.

1) *Performance Metrics*: The primary goal is to measure the model’s ability to predict the next token in a sequence. This is assessed using the following metrics:

- **Perplexity (PPL)**: This is the main metric reported for language modeling performance. It is derived from the cross-entropy loss and provides a more intuitive measure of how well the model’s probability distribution predicts a given sample. It is calculated using eq. (7) where $\mathcal{L}_{CE}$ is the average cross-entropy loss over the entire validation set. A lower perplexity score indicates better model performance. Conceptually, perplexity can be interpreted as the effective branching factor of the model; a perplexity score of N , for instance, suggests that the model is, on average, as uncertain as if it had to choose uniformly among N plausible candidates for the next token.

$$\text{PPL} = e^{\mathcal{L}_{CE}} \quad (7)$$

Perplexity is widely used in evaluation of LLMs including GPT-1 [1] and BERT [2].

- **Positional Loss Analysis**: To gain deeper insight into the model’s behavior, we analyze the cross-entropy loss across different positions in the input sequence. The context window is divided into segments of 64 tokens, and the average loss is calculated for each segment. This allows us to observe whether the model’s predictive accuracy degrades as the context grows longer, providing a more granular view of its long-range dependency handling.

2) *Efficiency Metrics*: A key aspect of our research is to create efficient models. Therefore, we also measure the following:

- **Model Size**: The total number of trainable parameters in the model. This provides a direct measure of the model’s memory footprint.
- **Computational Cost**: We calculate the total number of Floating Point Operations (FLOPs) required for a single forward pass, as proposed in scaling studies by Kaplan et al [5] and adopted in later work on compute-efficient models scaling [4], [13]. This provides a standardized, hardware-independent measure of the model’s computational complexity.

IV. RESULTS

This section presents the empirical results of our study. We begin by analyzing the impact of different maximum context lengths for small models, then establish performance baselines for classically trained models to analyze their scaling properties. We then provide a comparative analysis of models enhanced with teacher-student knowledge distillation. Finally, we evaluate the efficacy of self-distillation as a method for reducing the computational cost required to achieve a target pre-training loss.

A. Analysis of Context Length Impact

One of the key aspects when comparing language models of different sizes is the selection of an appropriate context length. Intuitively, and as observed in the literature [15], larger models are required to effectively process longer contexts and understand long-term semantics. Smaller models, when faced with an excessively long context, may struggle to utilize the additional information, leading to performance degradation.

To ensure a fair comparison across the different model scales evaluated in our main study, we conducted a preliminary analysis. The goal was to determine whether smaller models could effectively handle a context length of 1024 tokens, which was intended for larger models. For this purpose, we ran experiments with 1M and 5M parameter models, training six different variants of each model with context lengths of 64, 128, 256, 512, 1024, and 2048 tokens. All models were trained on a single A100 GPU for 10,000 steps, and performance was measured as the average loss on the validation set.

The results for the 1M parameter model are shown in Fig. 1, which plots the average validation loss as a function of context length. It is evident that the validation loss decreases as the context length increases up to 512 tokens, after which it begins to increase again. This observation raised concerns that a fixed context length of 1024 tokens might unfairly penalize smaller models in comparisons.

To investigate whether the observed degradation was due to an insufficient number of model parameters, the same experiment was repeated with a 5M parameter model. The results, shown in Fig. 2, indicate that for this model, the validation loss remains stable or even continues to decrease

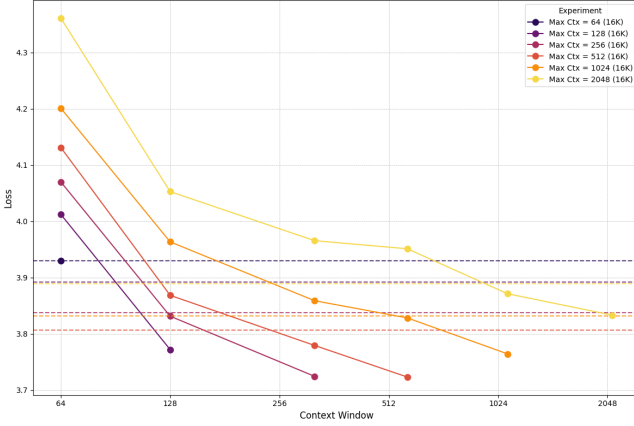


Fig. 1. Comparison of average loss for the 1M parameter model with different maximum context lengths

when moving from a context length of 512 to 1024 tokens. However, this model also experiences a performance drop, as the validation loss begins to increase at a context length of 2048 tokens.

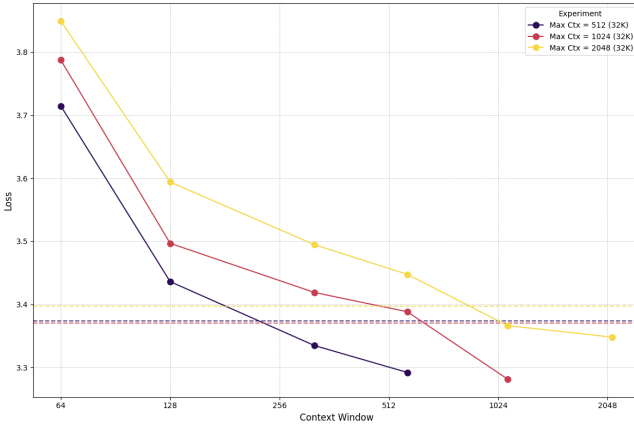


Fig. 2. Comparison of average loss for the 5M parameter model with different maximum context lengths

These results indicate that a context length of 1024 tokens is suitable for models of 5M parameters and larger, while smaller models show optimal performance up to a context length of 512 tokens. Adopting this tailored approach is crucial for mitigating potential biases in our main study, ensuring that observed performance differences can be attributed to the training strategies themselves, rather than to a suboptimal context length.

B. Baseline Performance and Scaling Laws

The initial phase of our research focused on establishing robust performance baselines. We trained seven models of varying sizes using a classical training procedure. The training loss for each model, plotted against the number of training steps, is presented in Fig. 3. As illustrated in Fig. 3, there is a clear and consistent relationship between model size and performance. For the same number of training

steps (12,000), larger models consistently achieve a lower validation loss. This confirms the fundamental principle that model scale is a primary driver of performance.



Fig. 3. Training loss curves for classically trained models of varying sizes

C. Comparative Analysis of Knowledge Distillation

After establishing the baselines, we investigated the effectiveness of the classical teacher-student knowledge distillation paradigm. The objective was to determine if a smaller student model could achieve superior performance when guided by a larger, pre-trained teacher model.

As shown in Fig. 4, a clear performance gap exists between the classically trained 5M model and its much larger 30M teacher, which serves as the performance benchmark. The key finding from this experiment is that both knowledge distillation methods provide a slight advantage in the initial phase of training.

The loss curves for both distilled variants are positioned below the 5M baseline during the first several thousand steps. A closer look at the initial phase reveals that while logits-based distillation provides a brief initial advantage, both methods quickly converge to a similar performance trajectory for the remainder of the training phase. However, this initial performance gap between the distilled models and the classic baseline begins to narrow as training progresses. Towards the end of the training run, all three 5M models converge to a similar final training loss value.



Fig. 4. Training loss comparison for 5M student and 30M teacher models

In further experiment, we investigated whether knowledge distillation can accelerate learning even when the teacher

and student models are of the same size. Fig. 5 presents the training loss curves for a 30M parameter model trained classically, compared against two variants of the same model distilled from a pre-trained, frozen 30M teacher using either hidden states or logits.

A similar pattern of accelerated initial learning is observed in this experiment. During the initial phase, both distilled models exhibit a lower training loss than the classically trained baseline. Notably, hidden state distillation maintains a consistent advantage over logits distillation until approximately the 9,000-step mark, after which their performance curves merge. However, the initial overall advantage against the baseline narrows as training progresses, with all models ultimately converging to a nearly identical final loss value.



Fig. 5. Training loss comparison for 30M student and 30M teacher models

D. Evaluating the Efficacy of Self-Distillation

The final and central phase of our research evaluates whether a model can accelerate its own learning through self-distillation. Here, a model learns from a frozen, checkpoint of itself on step 3000. Fig. 6 presents the results of this experiment for the 30M parameter model.

In the initial phase of training, up to approximately 3,000 steps, the distilled model is trained classically. At step 3000, the model weights are frozen and initialized as a separate teacher model. However, after this point, a divergence becomes apparent: the training loss for the self-distilled model begins to rise above that of the classically trained model. For the remainder of the training run, the self-distilled model maintains a consistently higher training loss compared to the classically trained model. These results differ from experiments with convolutional neural networks, where it has been shown that self-distillation can serve as a regularizer during training.

To our knowledge, no prior study tests self-distillation on decoder-only Transformers across this parameter range.

V. DISCUSSION

Our empirical results provide a nuanced perspective on the role of knowledge distillation in training language models. This section discusses the key implications of our findings, focusing on the distinct behaviors of classical teacher-student distillation and self-distillation, and addresses the unexpected outcomes that challenge our initial hypothesis.

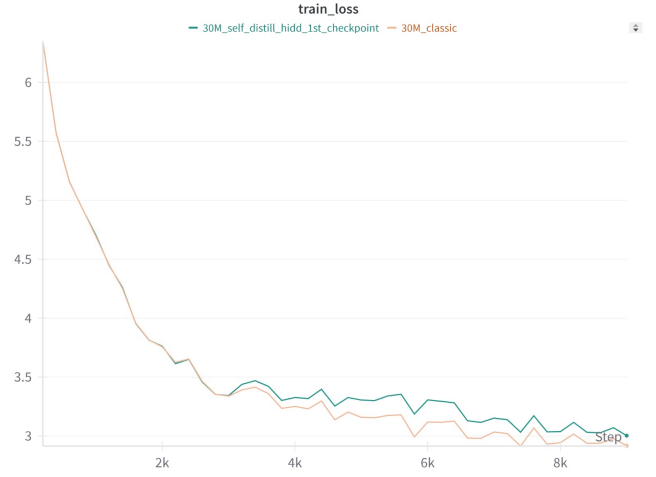


Fig. 6. Training loss comparison for 30M model with self-distillation

A. The Role of Knowledge Distillation as a Convergence Accelerator

Our experiments with the classical teacher-student paradigm consistently demonstrate that knowledge distillation serves as a powerful convergence accelerator. As shown in both the 5M and 30M model experiments (Fig. 4 and Fig. 5), students guided by a teacher achieve a lower training loss significantly faster than their classically trained counterparts. This initial boost suggests that the teacher model effectively transfers foundational patterns and representations it has already learned, allowing the student to bypass the earliest, most difficult stages of learning.

However, a crucial observation is that this initial advantage diminishes as training progresses. The convergence of the distilled models' loss curves with the baseline towards the end of the training run suggests that there is a fundamental performance ceiling dictated by the student model's own parameter capacity. The teacher can guide the student to this ceiling more efficiently, but it cannot help it break through it. This implies that for a given model architecture and dataset, knowledge distillation is primarily a tool for improving training efficiency (i.e., reducing the time or compute needed to reach a target performance) rather than a method for unlocking a higher absolute performance level than the model's capacity allows.

B. Failure of Self-Distillation and Potential Causes

The self-distilled 30M model not only failed to accelerate but began to perform worse than the classically trained baseline after an initial period of parity (Fig. 6). This negative result is highly informative and points to the inherent complexities and sensitivities of intra-model knowledge transfer. We hypothesize several potential causes for this outcome:

- 1) **Premature Teacher Checkpoint:** The teacher in our self-distillation experiment was a checkpoint from an early stage of training. It is plausible that this teacher was not yet mature enough; its internal representations may have been noisy, suboptimal, or unstable. Forcing

the student model to mimic these flawed representations could have been actively detrimental, leading it down a suboptimal optimization path.

- 2) **Destructive Interference Between Losses:** The self-distillation process introduces a secondary objective (e.g., MSE loss on hidden states) that competes with the primary causal language modeling objective (cross-entropy loss). It is possible that these two objectives were in conflict. The pressure to minimize the distillation loss might have pulled the model’s weights in a direction that was unhelpful for the ultimate goal of predicting the next token, resulting in a higher overall training loss.
- 3) **Incompatible Layer Pairing:** The choice of which layers serve as teacher and student is a critical, yet poorly understood, hyperparameter. The semantic representations of a very deep layer may be too abstract or different from those of a very shallow layer, making the direct minimization of their difference a poor training signal.

C. Limitations and Directions for Future Work

Our study, while comprehensive in its analysis of model scales, has several limitations that open avenues for future research. Our findings are based on a single dataset (Wikitext-103) and a specific implementation of self-distillation using a single, early-stage checkpoint.

Based on our results, future work should focus on systematically investigating the conditions under which self-distillation can succeed. Key research questions include:

- At what point during training does a model become a mature and effective teacher for itself?
- How can the pairing of teacher and student layers be optimized to ensure compatible knowledge transfer?
- Could an adaptive weighting scheme for the distillation loss mitigate the problem of destructive interference?

Answering these questions would provide a clearer understanding of the self-distillation mechanism and could unlock its potential as a powerful tool for efficient model training.

VI. CONCLUSION

The central aim of this work was to investigate whether self-distillation can be an effective technique for reducing the computational cost necessary to achieve a target pre-training loss in language models. We approached this by empirically comparing three training strategies: a classical baseline, a teacher-student knowledge distillation framework, and a self-distillation mechanism where a model learns from a previous checkpoint of itself.

Our results confirmed several key behaviors. First, our baseline models successfully replicated established scaling laws. Second, we demonstrated that classical knowledge distillation serves as a powerful convergence accelerator, although the ultimate performance is limited by the student model’s own parameter capacity. However, our primary hypothesis concerning self-distillation was not validated. The implemented self-distillation approach not only failed

to accelerate learning but led to performance degradation compared to the baseline model.

REFERENCES

- [1] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training,” OpenAI, Tech. Rep. GPT-1, 2018.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [3] M. Shueybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” *arXiv preprint arXiv:1909.08053*, 2019.
- [4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language Models are Few-Shot Learners,” *arXiv preprint arXiv:2005.14165*, 2020.
- [5] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling Laws for Neural Language Models,” *arXiv preprint arXiv:2001.08361*, 2020.
- [6] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” *arXiv preprint arXiv:1910.01108*, 2019.
- [7] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, and Q. Liu, “TinyBERT: Distilling BERT for Natural Language Understanding,” *arXiv preprint arXiv:1909.10351*, 2019.
- [8] Y. Gu, L. Dong, F. Wei, and M. Huang, “MiniLLM: Knowledge Distillation of Large Language Models,” *arXiv preprint arXiv:2306.08543*, 2024.
- [9] W. Yuan, R. Y. Pang, K. Cho, X. Li, S. Sukhbaatar, J. Xu, and J. Weston, “Self-Rewarding Language Models,” *arXiv preprint arXiv:2401.10020*, 2024.
- [10] C. Burns, P. Izmailov, J. H. Kirchner, B. Baker, L. Gao, L. Aschenbrenner, Y. Chen, A. Ecoffet, M. Joglekar, J. Leike, I. Sutskever, and J. Wu, “Weak-to-Strong Generalization: Eliciting Strong Capabilities With Weak Supervision,” *arXiv preprint arXiv:2312.09390*, 2023.
- [11] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” *arXiv preprint arXiv:1706.03762*, 2023.
- [12] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and Memory-Efficient Exact attention with IO-Awareness,” *arXiv preprint arXiv:2205.14135*, 2022.
- [13] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” *arXiv preprint arXiv:1910.10683*, 2023.
- [14] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. d. L. Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Milican, G. v. d. Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre, “Training Compute-Optimal Large Language Models,” *arXiv preprint arXiv:2203.15556*, 2022.
- [15] J. Shi, Q. Ma, H. Liu, H. Zhao, J.-N. Hwang, and L. Li, “Explaining Context Length Scaling and Bounds for Language Models,” *arXiv preprint arXiv:2502.01481*, 2025.

APPENDIX I SUPPLEMENTAL FIGURES FOR LEARNING RATE SCHEDULE

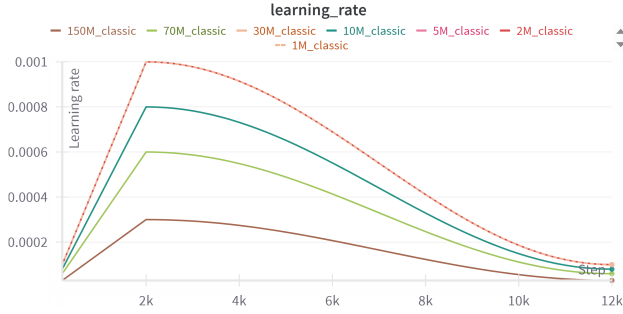


Fig. 7. Learning rate schedule of classically trained models

APPENDIX II SUPPLEMENTAL FIGURES FOR CONTEXT LENGTH IMPACT ANALYSIS

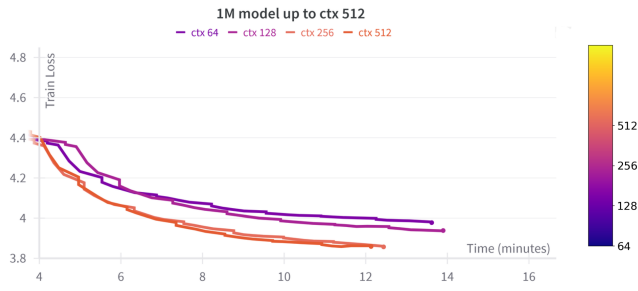


Fig. 8. Training results for the 1M parameter model up to a context length of 512 tokens

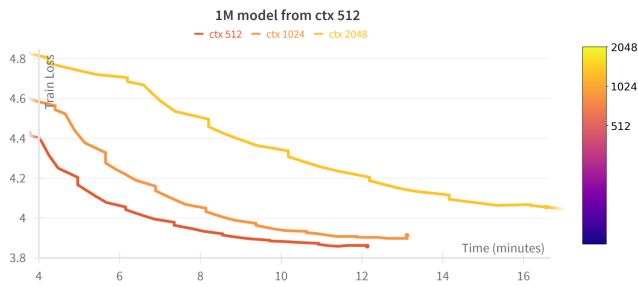


Fig. 9. Training results for the 1M parameter model with context lengths of 512, 1024, and 2048 tokens

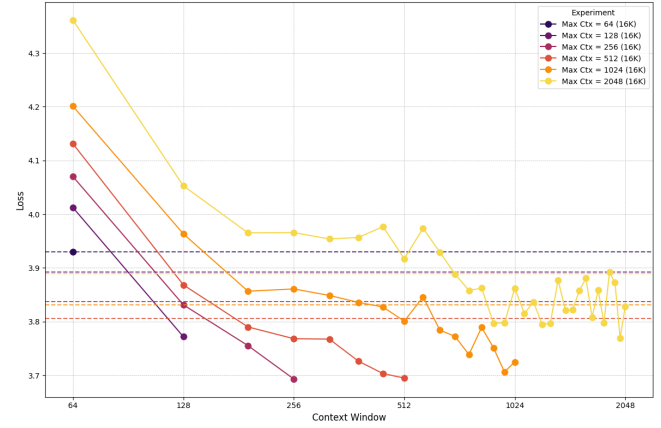


Fig. 10. Full comparison of average loss for the 1M parameter model with different maximum context lengths

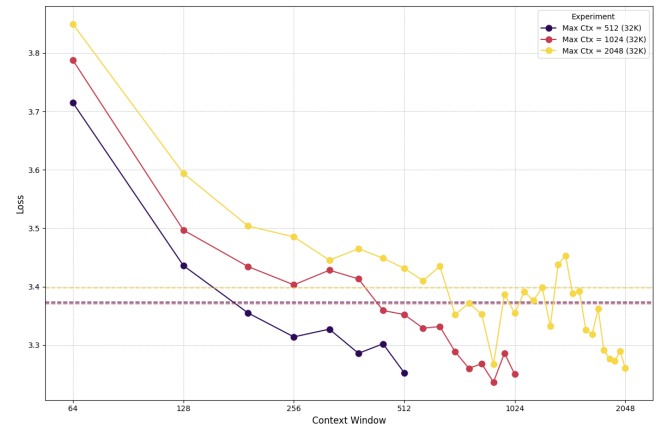


Fig. 11. Full comparison of average loss for the 5M parameter model with different maximum context lengths

APPENDIX III SUPPLEMENTAL FIGURES FOR LEARNING RATE SENSITIVITY ANALYSIS

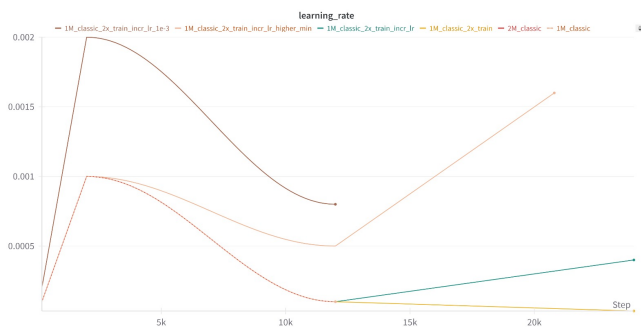


Fig. 12. Learning rate schedules for the sensitivity analysis

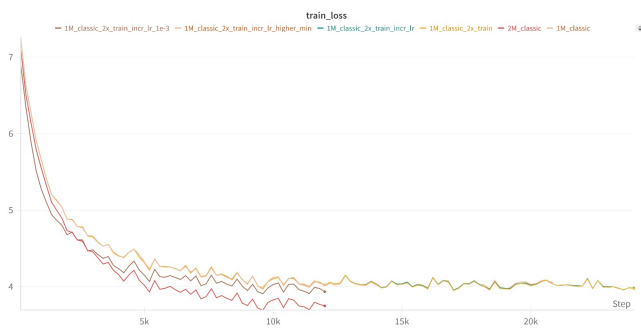


Fig. 13. Training loss convergence for different learning rate schedules